MSK: A Formal Specification of the Multimedia System Kernel

An *everything-is-a-node* store: absolute existence, operations, and the interface to contextual existence

Specification draft v0.1.2

4 November 2026

Abstract

We specify MSK (Multimedia System Kernel), a data store for content production in which every piece of information (a cut number, an assignee, an asset, a state) is held as a node of a content-addressed graph. Presentation structures such as progress tables, dependency graphs and workload views are not stored: they are *contextual* existences derived from a small core of *absolute* existences. The specification fixes the absolute layer completely (Access, Field, Node, Container and six operations) and fixes only the *interface* of the contextual layer (evaluation of Abstract Edges and graph nodes, and the firing conditions of Directional Edge runtimes). We prove that every operation preserves well-formedness and derive the properties on which the contextual layer relies. Points on which the source material is silent are *not* decided silently: they are tagged **[OD-n]** in the text and collected, with their provisional treatment, in Section 8.

Contents

1 Introduction, scope and conventions **3**
- 1.1 Purpose . . . 3
- 1.2 Scope . . . 3
- 1.3 Conventions . . . 3
- 1.4 Provenance of the design . . . 3

2 Basic domains and parameters **3**

3 Absolute existence: the state space **4**
- 3.1 Components . . . 4
- 3.2 Read-only primitives . . . 5
- 3.3 Well-formedness . . . 5

4 Operations **6**
- 4.1 Transition format . . . 6
- 4.2 Interning . . . 6
- 4.3 Field operations . . . 6
- 4.4 Data operations . . . 7
- 4.5 Executions . . . 8

5 Contextual existence: interface **8**
- 5.1 Abstract Edges and Tables . . . 8
- 5.2 Graph nodes with subnodes . . . 9
- 5.3 Directional Edges, conditions and runtimes . . . 9

6 Metatheory 9

7 Worked example 12

8 Decision register 12

A Summary of operations 14

B Index of well-formedness conditions 14

1 Introduction, scope and conventions

1.1 Purpose

Production data ("cut number: 1", "assignee: Yamada", "asset: model", "state: in progress") is stored as nodes. The same stored data can be arranged as a progress table (cut → asset, assignee, state), as a dependency graph (cut → asset_1 → asset_2), or as a workload view (assignee → asset → state), without storing any of these arrangements. The specification separates what is stored (*absolute existence*: Field, Node, Container, Access) from what is derived from it (*contextual existence*: Edge, Table, Graph).

1.2 Scope

In scope. (i) The state space of the absolute layer and its well-formedness conditions; (ii) the operations defField, addDef, dropDef, write, update, retarget; (iii) read-only primitives (*read*, *referrers*, *match*) that are sufficient to implement the contextual layer; (iv) the evaluation of Abstract Edges and graph nodes with subnodes, and the firing condition of Directional Edge runtimes; (v) metatheory.

Out of scope (implementation-defined). (a) Edge servers, replication and merging; (b) physical storage, indexing, caching and garbage collection of unreferenced addresses; (c) the virtual file system (VFS) layer, including the policy by which files change (**[OD-7]**); (d) path-style access of the form `graph:path`; (e) access control; (f) the language, scheduling and termination of edge runtimes; (g) rendering of tables and graphs, and the storage format of contextual definitions (**[OD-8]**, **[OD-9]**).

1.3 Conventions

For sets X, Y, $X \rightharpoonup_{\text{fin}} Y$ is the set of finite partial functions. For $f : X \rightharpoonup_{\text{fin}} Y$: $\operatorname{dom} f$ and $\operatorname{rng} f$ are its domain and range, and $\operatorname{graph} f = \{(x, f(x)) : x \in \operatorname{dom} f\}$; we identify f with $\operatorname{graph} f$ when convenient. The *override* $f \oplus g$ is the partial function with domain $\operatorname{dom} f \cup \operatorname{dom} g$ that equals $g(x)$ on $\operatorname{dom} g$ and $f(x)$ elsewhere; $f[x \mapsto y] := f \oplus \{x \mapsto y\}$. $\mathcal{P}_{\text{fin}}(X)$ denotes the finite subsets of X. $\perp$ denotes "undefined". Pairs written $\langle v : \tau \rangle$ are data (value v of type τ).

Statements typeset as Definition, Assumption and Operation are normative. Remarks and Examples are informative. Theorems are consequences of the normative text.

1.4 Provenance of the design

The decisions fixed by the specification's author are listed in Table 1; choices that the source material left open are listed in Table 2 and tagged **[OD-n]** where they are used.

2 Basic domains and parameters

Definition 2.1 (Universes)**.** Fix a finite alphabet Σ and non-empty sets $\mathcal{P}$ (property names), $\mathcal{F}$ (field names) and $\mathcal{T}$ (types, see below). *Values* are character strings, $\mathcal{V} := \Sigma^*$: every value is stored as a string and the store gives it no further meaning. Let $\mathsf{Path} \subseteq \mathcal{V}$ (relative file paths, interpreted relative to the VFS directory `/data`) and $\mathsf{Prog} \subseteq \mathcal{V}$ (expression programs in some implementation language). A *type* is a user-defined category of data such as person, cut or status. It is an opaque label: it has no relation to programming-language types such as integers or strings, carries no operations, and is used only for equality tests. $\mathcal{T}$ contains two distinct reserved types: ρ (*Reference*) and ε (*Expression*) **[OD-11]**. Data are pairs $\mathcal{D} := \mathcal{V} \times \mathcal{T}$ with projections $d.\mathsf{value}$ and $d.\mathsf{type}$. A datum d is *admissible* if $d.\mathsf{type} = \rho \Rightarrow d.\mathsf{value} \in \mathsf{Path}$ and $d.\mathsf{type} = \varepsilon \Rightarrow d.\mathsf{value} \in \mathsf{Prog}$. Let $\mathcal{D}^{\text{ok}} \subseteq \mathcal{D}$ be the admissible data.

Remark 2.2 (Strings and categories). Because values are strings and the type is part of a datum, "Yamada" as a person and "Yamada" as an asset are different data with different addresses, and the strings "1" and "01" are different values. Any numeric, ordering or other interpretation of a string belongs to the contextual layer. Two nodes share an address only if both the string and the category coincide; hence a bulk edit by retarget affects only nodes of that category.

Definition 2.3 (Addresses and identifiers)**.** $\mathbb{A} := \{0,1\}^{128}$ is the set of addresses (written as 32 hexadecimal digits). The *null address* is $\mathbf{0} := 0^{128}$ (0x00...0); $\mathbb{A}^+ := \mathbb{A} \setminus \{\mathbf{0}\}$. $\mathbb{I} := \{0,1\}^{128}$ is the set of container identifiers. $\mathbb{A}$ and $\mathbb{I}$ are distinct sorts: no operation compares an element of one with an element of the other.

Definition 2.4 (Address function)**.** A total function $h : \mathcal{D} \to \mathbb{A}$ is fixed for the lifetime of a store and is identical on all replicas. The source proposal instantiates h as a 128-bit digest (MD5) of a canonical encoding of the pair (value, type) **[OD-10]**.

Remark 2.5. Collision-freedom of h is *not* assumed (it cannot hold on an infinite domain). Instead, the operations below fail (result Err) whenever an address would be assigned to two different data (Definition 4.1), so well-formedness (WF1) holds unconditionally.

Definition 2.6 (File environment)**.** A *file environment* is a partial function $\mathsf{fs} : \mathsf{Path} \rightharpoonup \mathsf{Bytes}$ managed by the VFS. It is *not* part of the store state (Definition 3.5) and is not constrained by this specification (**[OD-7]**).

3 Absolute existence: the state space

Absolute existences are *Access*, *Field*, *Node* and *Container*. They are pure data and carry no directional meaning.

3.1 Components

Definition 3.1 (Access)**.** An *Access map* is a finite partial function $\mathsf{Acc} : \mathbb{A}^+ \rightharpoonup_{\text{fin}} \mathcal{D}$. $\mathsf{Acc}(a) = d$ means that the datum d (value and type) is stored at address a. The address $\mathbf{0}$ is never in $\operatorname{dom}\mathsf{Acc}$; it denotes *null*.

Definition 3.2 (Field)**.** A *field definition* is a finite partial function $\delta \in \mathsf{Def} := \mathcal{P} \rightharpoonup_{\text{fin}} \mathcal{T}$; the pairs $(p,\tau) \in \operatorname{graph}\delta$ are the elements $\mathsf{def}(x)_i = (\mathsf{property}, \mathsf{type})$ of the source. A *Field map* is $\Phi : \mathcal{F} \rightharpoonup_{\text{fin}} \mathsf{Def}$. We write $\mathsf{def}(x) := \Phi(x)$. Reading a field definition as a partial function means that a property has at most one type within a field **[OD-5]**.

Definition 3.3 (Node, content)**.** A *node* is a pair $(p,a) \in \mathsf{Node} := \mathcal{P} \times \mathbb{A}$ (a property and an address). A *content* is a finite partial function $c \in \mathsf{Cnt} := \mathcal{P} \rightharpoonup_{\text{fin}} \mathbb{A}$, with node set $\operatorname{graph} c$. Reading a content as a partial function means that a container holds at most one node per property **[OD-5]**.

Definition 3.4 (Container)**.** A *Container map* is $\Gamma : \mathbb{I} \rightharpoonup_{\text{fin}} \mathcal{F} \times \mathsf{Cnt}$. For $n \in \operatorname{dom}\Gamma$ with $\Gamma(n) = (x,c)$ we write $\mathsf{fld}(n) := x$ (the field the container belongs to) and $\mathsf{cnt}(n) := c$. Container identifiers are arbitrary, unique, and unrelated to content (in implementations: random).

Definition 3.5 (State)**.** A *state* is a triple $s = (\Phi, \mathsf{Acc}, \Gamma)$; its components are written $s.\Phi$, $s.\mathsf{Acc}$, $s.\Gamma$, and the accessors $\mathsf{fld}_s, \mathsf{cnt}_s$ refer to $s.\Gamma$. The *empty state* is $s_\emptyset = (\emptyset, \emptyset, \emptyset)$. The state contains no log, version or history component (see Remark 4.12). Let State be the set of states.

3.2 Read-only primitives

Definition 3.6 (Read)**.** For a state s, $n \in \mathbb{I}$ and $p \in \mathcal{P}$:

$$\mathsf{rd}_s(n,p) := \begin{cases} \mathsf{cnt}_s(n)(p) & \text{if } n \in \operatorname{dom} s.\Gamma \text{ and } p \in \operatorname{dom} \mathsf{cnt}_s(n), \\ \mathbf{0} & \text{otherwise.} \end{cases}$$

rd_s is total. In particular a property that is defined by the field but absent from the container reads as null, as does any property absent from the container. The *datum read* is $\mathsf{dat}_s(n,p) := s.\mathsf{Acc}(\mathsf{rd}_s(n,p))$ if $\mathsf{rd}_s(n,p) \neq \mathbf{0}$ and nil otherwise.

Definition 3.7 (Effective content)**.** $\mathsf{eff}_s(n) := \{(p,a) \in \operatorname{graph} \mathsf{cnt}_s(n) : a \neq \mathbf{0}\}$ for $n \in \operatorname{dom} s.\Gamma$, and $\mathsf{eff}_s(n) := \emptyset$ otherwise. For a content c we write $\mathsf{eff}(c) := \{(p,a) \in \operatorname{graph} c : a \neq \mathbf{0}\}$. Thus a node with the null address is indistinguishable from an absent node **[OD-2]**.

Lemma 3.8. *For all s, n, p and $a \neq \mathbf{0}$: $\mathsf{rd}_s(n,p) = a \iff (p,a) \in \mathsf{eff}_s(n)$. Consequently $\mathsf{rd}_s(n,p) = \mathbf{0} \iff$ there is no a with $(p,a) \in \mathsf{eff}_s(n)$.*

Proof. Immediate from Definitions 3.6 and 3.7. □

Definition 3.9 (Overrides)**.** For $n \in \operatorname{dom} s.\Gamma$: $\mathsf{ovr}_s(n) := \operatorname{dom} \mathsf{cnt}_s(n) \setminus \operatorname{dom} \Phi(\mathsf{fld}_s(n))$, the properties that the container carries but its field does not define (per-container exceptions).

Definition 3.10 (Referrers)**.** For $a \in \mathbb{A}$:

$$\mathsf{refs}_s(a) := \{(n,p) : n \in \operatorname{dom} s.\Gamma,\ p \in \operatorname{dom} \mathsf{cnt}_s(n),\ \mathsf{cnt}_s(n)(p) = a\}.$$

Definition 3.11 (Match)**.** For a finite set of nodes $K \subseteq \mathsf{Node}$: $\mathsf{Match}_s(K) := \{n \in \operatorname{dom} s.\Gamma : \forall (p,b) \in K.\ \mathsf{rd}_s(n,p) = b\}$.

Lemma 3.12. *(i) $\mathsf{Match}_s(\emptyset) = \operatorname{dom} s.\Gamma$. (ii) $K \subseteq K' \Rightarrow \mathsf{Match}_s(K') \subseteq \mathsf{Match}_s(K)$. (iii) If $K \subseteq \mathcal{P} \times \mathbb{A}^+$ then $n \in \mathsf{Match}_s(K) \iff K \subseteq \mathsf{eff}_s(n)$.*

Proof. (i) and (ii) are immediate from the universal quantifier in Definition 3.11; (iii) follows from Lemma 3.8. □

Definition 3.13 (Reference resolution)**.** Given a file environment fs and $a \in \operatorname{dom} \mathsf{Acc}$ with $\mathsf{Acc}(a) = \langle \pi : \rho \rangle$: $\mathsf{resolve}_{\mathsf{fs}}(a) := \mathsf{fs}(\pi)$ if $\pi \in \operatorname{dom} \mathsf{fs}$, and $\perp$ otherwise. In the latter case the reference is said to have *null linkage*: the datum $\langle \pi : \rho \rangle$ and every node pointing to it persist unchanged.

3.3 Well-formedness

Definition 3.14 (Well-formed state)**.** A state $s = (\Phi, \mathsf{Acc}, \Gamma)$ is *well-formed*, written $\mathrm{WF}(s)$, iff:

WF1 *(address integrity)* $\forall a \in \operatorname{dom} \mathsf{Acc}.\ h(\mathsf{Acc}(a)) = a$.

WF2 *(admissible data)* $\forall a \in \operatorname{dom} \mathsf{Acc}.\ \mathsf{Acc}(a) \in \mathcal{D}^{\mathrm{ok}}$.

WF3 *(closure)* $\forall n \in \operatorname{dom} \Gamma,\ p \in \operatorname{dom} \mathsf{cnt}(n).\ \mathsf{cnt}(n)(p) \in \operatorname{dom} \mathsf{Acc} \cup \{\mathbf{0}\}$.

WF4 *(field existence)* $\forall n \in \operatorname{dom} \Gamma.\ \mathsf{fld}(n) \in \operatorname{dom} \Phi$.

WF5 *(field coverage)* $\forall n \in \operatorname{dom} \Gamma.\ \operatorname{dom} \Phi(\mathsf{fld}(n)) \subseteq \operatorname{dom} \mathsf{cnt}(n)$, i.e. the properties of a field are included in the properties of each of its containers; the surplus is $\mathsf{ovr}(n)$ **[OD-3]**.

WF6 *(type conformance)* $\forall n \in \operatorname{dom} \Gamma,\ p \in \operatorname{dom} \Phi(\mathsf{fld}(n))$: if $a := \mathsf{cnt}(n)(p) \neq \mathbf{0}$ then $\mathsf{Acc}(a).\mathsf{type} = \Phi(\mathsf{fld}(n))(p)$.

Lemma 3.15. $\mathrm{WF}(s_\emptyset)$.

Proof. All six conditions are universally quantified over empty sets. □

4 Operations

4.1 Transition format

An operation instance op relates a state s to a state s' and a result r, written $s \xrightarrow{op\,/\,r} s'$. On failure $r = \mathsf{Err}$ and $s' = s$ (failed operations have no effect). The relation is non-deterministic in two respects only: the choice of a fresh container identifier, and the candidate selection function sel of write ([OD-1]). A step $s \to s'$ is *successful* if $s \xrightarrow{op\,/\,r} s'$ for some op and some $r \neq \mathsf{Err}$. The input alphabet of data is $U := \mathcal{D} \cup \{\mathsf{nil}\}$, where nil stands for "set to null". A *write input* is $W : \mathcal{P} \rightharpoonup_{\mathrm{fin}} U$.

4.2 Interning

Definition 4.1 (Interning)**.** Let Acc be an Access map and W a write input. Put $D_W := \{W(p) : p \in \operatorname{dom} W,\ W(p) \neq \mathsf{nil}\}$. $\mathsf{Intern}(\mathsf{Acc}, W) = \mathsf{Err}$ iff at least one of the following holds:

(i) some $d \in D_W$ is not admissible ($d \notin \mathcal{D}^{\mathrm{ok}}$);

(ii) some $d \in D_W$ has $h(d) = \mathbf{0}$;

(iii) some $d \in D_W$ has $h(d) \in \operatorname{dom} \mathsf{Acc}$ and $\mathsf{Acc}(h(d)) \neq d$ (collision with a stored datum);

(iv) some $d \neq d'$ in D_W have $h(d) = h(d')$ (collision within the input).

Otherwise $\mathsf{Intern}(\mathsf{Acc}, W) = (\mathsf{Acc}', \widehat{W})$ with

$$\mathsf{Acc}' := \mathsf{Acc} \oplus \{h(d) \mapsto d : d \in D_W\}, \qquad \widehat{W}(p) := \begin{cases} \mathbf{0} & W(p) = \mathsf{nil} \\ h(W(p)) & \text{otherwise} \end{cases} \quad (p \in \operatorname{dom} W).$$

For a single datum d we write $\mathsf{Intern}(\mathsf{Acc}, d)$ for $\mathsf{Intern}(\mathsf{Acc}, \{p_0 \mapsto d\})$ with a fixed dummy property p_0, and use the pair $(\mathsf{Acc}', h(d))$.

Lemma 4.2. *Suppose* $\mathsf{Intern}(\mathsf{Acc}, W) = (\mathsf{Acc}', \widehat{W})$. *Then (a)* $\operatorname{graph} \mathsf{Acc} \subseteq \operatorname{graph} \mathsf{Acc}'$ *(nothing is removed or changed); (b) if* Acc *satisfies WF1 and WF2 then so does* Acc'*; (c)* $\operatorname{rng} \widehat{W} \subseteq \operatorname{dom} \mathsf{Acc}' \cup \{\mathbf{0}\}$ *and* $\mathsf{Acc}'(\widehat{W}(p)) = W(p)$ *whenever* $W(p) \in \mathcal{D}$*; (d) the result does not depend on any ordering of* W*.*

Proof. (a) By (iii) every $d \in D_W$ with $h(d) \in \operatorname{dom} \mathsf{Acc}$ satisfies $\mathsf{Acc}(h(d)) = d$, so $\oplus$ only adds entries or rewrites an entry by itself. (b) New entries $h(d) \mapsto d$ satisfy $h(\mathsf{Acc}'(h(d))) = h(d)$ and $d \in \mathcal{D}^{\mathrm{ok}}$ by (i); old entries are unchanged by (a). Moreover $h(d) \neq \mathbf{0}$ by (ii), so $\mathbf{0} \notin \operatorname{dom} \mathsf{Acc}'$. (c) By construction and (iv) the map $d \mapsto h(d)$ is injective on D_W. (d) Acc' and $\widehat{W}$ are defined by set comprehensions. □

Definition 4.3 (Conformance)**.** For a field definition δ and input W: $\mathsf{Conf}_\delta(W)$ iff for every $p \in \operatorname{dom} \delta \cap \operatorname{dom} W$, $W(p) = \mathsf{nil}$ or $W(p).\mathsf{type} = \delta(p)$. Properties outside $\operatorname{dom} \delta$ (overrides) are unconstrained [OD-4].

4.3 Field operations

Operation 4.4 ($\mathsf{defField}(x, \delta)$)**. Precondition** $x \in \mathcal{F} \setminus \operatorname{dom} s.\Phi$ and $\delta \in \mathsf{Def}$.

Effect $s' = (s.\Phi \oplus \{x \mapsto \delta\},\ s.\mathsf{Acc},\ s.\Gamma)$; $r = \mathsf{ok}$.

Failure $r = \mathsf{Err}$ otherwise.

Operation 4.5 ($\mathsf{addDef}(x, p, \tau)$)**. Precondition** $x \in \operatorname{dom} s.\Phi$, $p \notin \operatorname{dom} s.\Phi(x)$, and for every n with $\mathsf{fld}_s(n) = x$ and $p \in \operatorname{dom} \mathsf{cnt}_s(n)$: $\mathsf{cnt}_s(n)(p) = \mathbf{0}$ or $s.\mathsf{Acc}(\mathsf{cnt}_s(n)(p)).\mathsf{type} = \tau$ (an existing override must already conform to the new definition).

Effect $s'.\Phi = s.\Phi[x \mapsto s.\Phi(x)[p \mapsto \tau]]$, $s'.\mathsf{Acc} = s.\mathsf{Acc}$, and for every n with $\mathsf{fld}_s(n) = x$ and $p \notin \operatorname{dom} \mathsf{cnt}_s(n)$, $\mathsf{cnt}_{s'}(n) = \mathsf{cnt}_s(n)[p \mapsto \mathbf{0}]$; all other containers are unchanged. $r = \mathsf{ok}$. (Eager padding, **[OD-3]**.)

Failure $r = \mathsf{Err}$ otherwise.

Operation 4.6 ($\mathsf{dropDef}(x, p)$)**. Precondition** $x \in \operatorname{dom} s.\Phi$ and $p \in \operatorname{dom} s.\Phi(x)$.

Effect $s' = (s.\Phi[x \mapsto s.\Phi(x) \setminus \{p\}],\ s.\mathsf{Acc},\ s.\Gamma)$, where $\setminus$ removes the key p. $r = \mathsf{ok}$. Existing nodes for p are retained and become overrides.

Failure $r = \mathsf{Err}$ otherwise.

4.4 Data operations

Assumption 4.7 (Candidate selection)**.** A function $\mathsf{sel} : \mathcal{P}_{\mathrm{fin}}(\mathbb{I}) \setminus \{\emptyset\} \to \mathbb{I}$ with $\mathsf{sel}(C) \in C$ is a parameter of the specification. Every result below holds for every such sel **[OD-1]**.

Operation 4.8 ($\mathsf{write}(x, W)$)**.** Creates a container in field x, or extends an existing one that carries no more information.

Precondition $x \in \operatorname{dom} s.\Phi$; $\mathsf{Conf}_{s.\Phi(x)}(W)$; and $\mathsf{Intern}(s.\mathsf{Acc}, W) = (\mathsf{Acc}', \widehat{W}) \neq \mathsf{Err}$.

Padding $\overline{W} := \widehat{W} \ \cup\ \{(p, \mathbf{0}) : p \in \operatorname{dom} s.\Phi(x) \setminus \operatorname{dom} \widehat{W}\} \in \mathsf{Cnt}$.

Candidates $\mathsf{Cand}_s(x, W) := \{n \in \operatorname{dom} s.\Gamma : \mathsf{fld}_s(n) = x \ \wedge\ \mathsf{eff}_s(n) \subseteq \mathsf{eff}(\overline{W})\}$ **[OD-6]**.

Effect, $\mathsf{Cand} = \emptyset$ choose a fresh $n \in \mathbb{I} \setminus \operatorname{dom} s.\Gamma$; $s' = (s.\Phi,\ \mathsf{Acc}',\ s.\Gamma \oplus \{n \mapsto (x, \overline{W})\})$; $r = n$.

Effect, $\mathsf{Cand} \neq \emptyset$ let $n_0 := \mathsf{sel}(\mathsf{Cand}_s(x, W))$; $s' = (s.\Phi,\ \mathsf{Acc}',\ s.\Gamma \oplus \{n_0 \mapsto (x, \overline{W})\})$; $r = n_0$.

Failure $r = \mathsf{Err}$ otherwise.

In words: if some existing container of x states only facts that the new content also states, that container is made equal to the new content; otherwise a new container is made. Data that were not yet indexed receive new addresses (Definition 4.1).

Operation 4.9 ($\mathsf{update}(n, p, u)$)**.** Changes one property of one container.

Precondition $n \in \operatorname{dom} s.\Gamma$; $u \in U$; with $x := \mathsf{fld}_s(n)$ and $W := \{p \mapsto u\}$: $\mathsf{Conf}_{s.\Phi(x)}(W)$ and $\mathsf{Intern}(s.\mathsf{Acc}, W) = (\mathsf{Acc}', \widehat{W}) \neq \mathsf{Err}$.

Effect $s' = (s.\Phi,\ \mathsf{Acc}',\ s.\Gamma \oplus \{n \mapsto (x,\ \mathsf{cnt}_s(n)[p \mapsto \widehat{W}(p)])\})$; $r = \mathsf{ok}$.

Failure $r = \mathsf{Err}$ otherwise.

If $p \notin \operatorname{dom} s.\Phi(x)$ the update creates or changes an override. The previous address of (n, p) is not retained (Remark 4.12).

Operation 4.10 ($\mathsf{retarget}(a, d)$)**.** Edits the datum behind an address for *all* nodes that refer to it, by moving them to the address of the new datum. The old address and its datum are kept.

Precondition $a \in \operatorname{dom} s.\mathsf{Acc}$; $d \in \mathcal{D}$; $\mathsf{Intern}(s.\mathsf{Acc}, d) = (\mathsf{Acc}', a') \neq \mathsf{Err}$ (so $a' = h(d)$); and for every $(n, p) \in R := \mathsf{refs}_s(a)$ with $p \in \operatorname{dom} s.\Phi(\mathsf{fld}_s(n))$: $d.\mathsf{type} = s.\Phi(\mathsf{fld}_s(n))(p)$.

Effect $s' = (s.\Phi,\ \mathsf{Acc}',\ \Gamma')$ where, simultaneously for all $n \in \operatorname{dom} s.\Gamma$,

$$\Gamma'(n) = (\mathsf{fld}_s(n),\ \mathsf{cnt}_s(n) \oplus \{p \mapsto a' : (n, p) \in R\}),$$

and $r = a'$.

Failure $r = \mathsf{Err}$ otherwise.

Remark 4.11 (Editing a single node). Editing the value of one node only is $\mathsf{update}(n, p, d)$: the node is assigned the address of the new datum, which is created if it is not yet indexed. A bulk edit restricted to a subset of the referrers is the sequential composition of such updates; no scoped variant of retarget is needed.

Remark 4.12 (No history). Following the specification's author, a container does not retain the value that an update replaces: the state (Definition 3.5) has no component that records earlier addresses of a slot. After retarget, the old address stays in Acc (Lemma 6.2) but no longer has referrers, and nothing records which nodes used to refer to it.

Remark 4.13 (Not specified). The source specifies no operation that deletes a container or a field, renames a property, or removes an entry of Acc. None is introduced here **[OD-6]**. In particular the specification never removes an address from Acc; reclamation of unreferenced addresses is an implementation concern.

4.5 Executions

Definition 4.14 (Execution, reachable state)**.** An *execution* is a sequence $s_\emptyset = s_0 \to s_1 \to \cdots \to s_k$ of successful steps. A state is *reachable* if it is the last state of some execution.

5 Contextual existence: interface

Contextual existences are *Abstract Edges*, *Tables*, *Graphs* and *Directional Edges*. Their internal mechanisms, storage and rendering are implementation-defined. This section fixes only what the absolute layer must provide so that any implementation can realize them, and the meaning of firing conditions.

Definition 5.1 (Contextual objects, purity)**.** A *contextual object* κ has an *evaluation* $[\![\kappa]\!] : \mathsf{State} \to \mathsf{Out}$ such that: (P1) $[\![\kappa]\!](s)$ is determined by s and the definition of κ alone and has no effect on s; (P2) every element occurring in $[\![\kappa]\!](s)$ is a container identifier of $s.\Gamma$, a property, or an address or datum of $s.\mathsf{Acc}$ ("contextual data are references to absolute data"); (P3) the only way a contextual object can cause a change of stored data is by issuing operations of Section 4.

5.1 Abstract Edges and Tables

Definition 5.2 (Abstract Edge)**.** An *abstract edge* is a pair $\alpha = (K, p)$ where $K \subseteq \mathsf{Node}$ is a finite set of nodes (the co-members of a cell) and $p \in \mathcal{P}$ is the property of the cell x the edge determines. Its evaluation is

$$[\![\alpha]\!]_s := \{(n, \mathsf{rd}_s(n, p)) : n \in \mathsf{Match}_s(K)\}.$$

Thus, for every container n that includes K, the cell x takes the address that n holds for $x.\mathsf{property}$. The result may be empty, a singleton, or contain several containers (all matches are returned).

Definition 5.3 (Table (informative))**.** For a field x and columns $P = (p_1, \ldots, p_k)$ the canonical table is $\mathsf{Tab}_{x,P}(s) := \big(n,\ (\mathsf{rd}_s(n, p_i))_{i \le k}\big)_{n \in \mathrm{dom}\, s.\Gamma,\ \mathsf{fld}_s(n) = x}$. Whether overrides are displayed, and the layout, are implementation-defined. Each cell (n, p_i) is the value of the abstract edge $(\{(q, \mathsf{rd}_s(n, q)) : q \in Q\}, p_i)$ for any set Q of identifying columns whose combination is unique to n.

5.2 Graph nodes with subnodes

Definition 5.4 (Graph node)**.** A *graph node* is $g = (p, a, K)$, a node (p, a) together with a finite set $K \subseteq \mathsf{Node}$ of *subnodes*. Its evaluation is

$$[\![g]\!]_s := \mathsf{Match}_s(K \cup \{(p, a)\}), \qquad \mathsf{Slots}_s(g) := \{(n, p) : n \in [\![g]\!]_s\}.$$

All matches are returned; choosing among them is not part of the specification.

Remark 5.5. With $K = \emptyset$, $[\![g]\!]_s$ is the group of all containers whose property p has address a (e.g. every cut that is "in progress"). With $K = \{(\mathsf{cut_num}, a_1), (\mathsf{work}, a_{\mathrm{mod}})\}$ it is the set of "in progress" cells of the modelling work of cut 1. Grouping is by address equality (Theorem 6.3); no values are compared.

5.3 Directional Edges, conditions and runtimes

Definition 5.6 (Condition)**.** A *condition* is a decidable predicate $\varphi : \mathsf{State} \times \mathsf{State} \to \{\top, \bot\}$ on pairs of states. The syntax in which implementations express conditions is not specified **[OD-9]**.

Definition 5.7 (Directional Edge)**.** A *directional edge* is a tuple $e = (g_{\mathrm{src}}, g_{\mathrm{dst}}, \varphi_e, \nu_e)$ with graph nodes $g_{\mathrm{src}}, g_{\mathrm{dst}}$, a condition φ_e, and an *expression node* $\nu_e = (n_\nu, p_\nu)$, a slot that holds the edge's expression **[OD-9]**. The edge is *armed in* s iff $a := \mathsf{rd}_s(n_\nu, p_\nu) \neq \mathbf{0}$ and $\mathsf{Acc}(a).\mathsf{type} = \varepsilon$; then the *expression* of e in s is $\mathsf{expr}_s(e) := \mathsf{Acc}(a).\mathsf{value} \in \mathsf{Prog}$.

Definition 5.8 (Firing)**.** $\mathsf{Fire}(e, s, s')$ holds iff $s \to s'$ is a successful step, e is armed in s', and $\varphi_e(s, s')$ holds.

Definition 5.9 (Standard conditions)**.** For a graph node g:

$$\mathsf{Enter}(g)(s, s') :\iff [\![g]\!]_{s'} \setminus [\![g]\!]_s \neq \emptyset, \qquad \mathsf{Leave}(g)(s, s') :\iff [\![g]\!]_s \setminus [\![g]\!]_{s'} \neq \emptyset.$$

Conditions are closed under $\wedge, \vee, \neg$. Both edge-triggered conditions ("becomes satisfied", such as Enter) and level-triggered ones ("is satisfied after the step") are conditions in the sense of Definition 5.6.

Definition 5.10 (Runtime interface)**.** The *runtime* of an edge e is an implementation-defined, possibly partial, function $\mathsf{Rt}_e : \mathsf{State} \times \mathsf{State} \rightharpoonup \mathsf{Op}^*$ from a firing pair to a finite sequence of operations of Section 4. Its implementation language is not specified. Any effect of a runtime on stored data is the application of the returned operations. Ordering, atomicity of the batch, failure handling, cascading and termination are implementation-defined **[OD-9]**.

Remark 5.11 (Sufficiency). The primitives rd, refs, Match, successful steps and Fire suffice to build the three views of the motivation: a progress table is $\mathsf{Tab}_{x,P}$; a dependency graph is the set of directional edges between graph nodes; the workload of an assignee b is the cardinality of $\mathsf{Match}_s(\{(\mathsf{assignee}, b)\}) \setminus \mathsf{Match}_s(\{(\mathsf{status}, a_{\mathrm{done}})\})$. Bulk renames are $\mathsf{retarget}$ and use the reverse index refs.

Remark 5.12 (Reconstructibility, non-normative). If the definitions of contextual objects are stored outside the absolute state, replicas that exchange only absolute state cannot reproduce them. Implementations that replicate are therefore advised to store those definitions as containers of reserved fields **[OD-8]**.

6 Metatheory

Theorem 6.1 (Preservation of well-formedness)**.** *If* $\mathrm{WF}(s)$ *and* $s \xrightarrow{op\,/\,r} s'$ *then* $\mathrm{WF}(s')$*. Hence every reachable state is well-formed.*

Proof. If $r = \mathsf{Err}$ then $s' = s$. Otherwise by cases on *op*; throughout, Lemma 4.2 gives WF1 and WF2 for Acc' whenever interning occurs, and WF3 is checked on the changed slots (old slots point into $\operatorname{dom} s.\mathsf{Acc} \subseteq \operatorname{dom} \mathsf{Acc}'$). *defField.* Acc, Γ unchanged; $\operatorname{dom} \Phi$ grows and no container belongs to the new field, so WF4–WF6 hold. *addDef.* Acc is unchanged. WF5: every container of x now has p in its domain, by padding or because it already had it. WF6: for the new definition (p, τ) a container has either a null address (padding) or an address whose type is τ, by the precondition; other definitions are unchanged. *dropDef.* $\operatorname{dom} \Phi(x)$ shrinks and contents are unchanged, so WF5 and WF6 are inherited. *write.* The new or replaced content $\overline{W}$ has addresses in $\operatorname{dom} \mathsf{Acc}' \cup \{\mathbf{0}\}$ (Lemma 4.2(c)); $x \in \operatorname{dom} \Phi$ gives WF4; $\overline{W}$ contains all of $\operatorname{dom} \Phi(x)$ by padding, giving WF5; conformance gives WF6. Other containers are unchanged. *update.* Only (n, p) changes; $\operatorname{dom} \mathsf{cnt}(n)$ only grows, so WF5 holds; if $p \in \operatorname{dom} \Phi(x)$ then conformance gives WF6; otherwise p is an override and is unconstrained. *retarget.* The new address $a' = h(d)$ lies in $\operatorname{dom} \mathsf{Acc}'$ and differs from $\mathbf{0}$ by Definition 4.1(ii), giving WF3; domains of contents are unchanged, giving WF5; the precondition gives WF6 on the moved slots; other slots are unchanged. The last claim follows by induction on executions from $\mathrm{WF}(s_\emptyset)$. □

Lemma 6.2 (Access monotonicity)**.** *If* $s \xrightarrow{op\,/\,r} s'$ *then* $\operatorname{graph} s.\mathsf{Acc} \subseteq \operatorname{graph} s'.\mathsf{Acc}$.

Proof. Only write, update and retarget change Acc, and only through Intern; apply Lemma 4.2(a). □

Theorem 6.3 (Address–value coherence)**.** *Let* $\mathrm{WF}(s)$. *For all* $a, b \in \operatorname{dom} s.\mathsf{Acc}$: $a = b \iff s.\mathsf{Acc}(a) = s.\mathsf{Acc}(b)$. *Equivalently, for* $d, d' \in \operatorname{rng} s.\mathsf{Acc}$: $d = d' \iff h(d) = h(d')$. *Hence two nodes denote the same datum iff their addresses are equal, and group-by is computed by address comparison.*

Proof. ($\Rightarrow$) Acc is a function. ($\Leftarrow$) By WF1, $a = h(\mathsf{Acc}(a)) = h(\mathsf{Acc}(b)) = b$. For the equivalent form let $d = \mathsf{Acc}(a)$, $d' = \mathsf{Acc}(b)$; then $h(d) = a$ and $h(d') = b$ by WF1. □

Corollary 6.4 (Group-by)**.** *Let* $\mathrm{WF}(s)$, $n, m \in \operatorname{dom} s.\Gamma$ *and* $p \in \mathcal{P}$. *Then* $\mathsf{rd}_s(n, p) = \mathsf{rd}_s(m, p)$ *iff* $\mathsf{dat}_s(n, p) = \mathsf{dat}_s(m, p)$.

Proof. If both addresses are $\mathbf{0}$ both data are nil. If exactly one is $\mathbf{0}$ both sides are false. Otherwise apply Theorem 6.3. □

Theorem 6.5 (Reads)**.** *Let* $\mathrm{WF}(s)$. *Then* rd_s *is total with values in* $\operatorname{dom} s.\mathsf{Acc} \cup \{\mathbf{0}\}$*; and if* $p \in \operatorname{dom} s.\Phi(\mathsf{fld}_s(n))$ *then* $p \in \operatorname{dom} \mathsf{cnt}_s(n)$. *Absent and null-valued nodes are indistinguishable by* rd *and by* eff*; they can differ only on overrides, where they are not observable.*

Proof. Totality is Definition 3.6 together with WF3; the second claim is WF5; indistinguishability is Lemma 3.8. □

Theorem 6.6 (write)**.** *Let* $\mathrm{WF}(s)$, $s \xrightarrow{\mathsf{write}(x,W)\,/\,n} s'$ *with* $n \neq \mathsf{Err}$, *and* $C := \mathsf{Cand}_s(x, W)$. *Then:*

(a) $n \in \operatorname{dom} s'.\Gamma$, $\mathsf{fld}_{s'}(n) = x$ *and* $\mathsf{eff}_{s'}(n) = \{(p, h(W(p))) : W(p) \in \mathcal{D}\}$*; for* $p \in \operatorname{dom} W$, $\mathsf{rd}_{s'}(n, p) = \widehat{W}(p)$.

(b) If $C = \emptyset$ *then* $n \notin \operatorname{dom} s.\Gamma$ *and* $\operatorname{dom} s'.\Gamma = \operatorname{dom} s.\Gamma \cup \{n\}$*; otherwise* $n = \mathsf{sel}(C) \in \operatorname{dom} s.\Gamma$ *and* $\operatorname{dom} s'.\Gamma = \operatorname{dom} s.\Gamma$.

(c) $s'.\Gamma(m) = s.\Gamma(m)$ *for all* $m \neq n$.

(d) (information monotonicity) $\mathsf{eff}_s(m) \subseteq \mathsf{eff}_{s'}(m)$ *for every* $m \in \operatorname{dom} s.\Gamma$.

(e) If a container of x *already has effective content equal to* $\mathsf{eff}(\overline{W})$*, no new container is created.*

Proof. (a) $\mathsf{eff}(\overline{W})$ drops exactly the null entries of $\overline{W}$, i.e. the nil inputs and the padding. (b),(c) are the two cases of the effect. (d) For $m \neq n$ by (c); for $m = n \in C$ by the definition of C; a new container has no predecessor. (e) Such a container belongs to C, so $C \neq \emptyset$. □

Theorem 6.7 (update)**.** *Let* $\mathrm{WF}(s)$ *and* $s \xrightarrow{\mathit{update}(n,p,u)\,/\,\mathsf{ok}} s'$. *Then* $\operatorname{dom} s'.\Gamma = \operatorname{dom} s.\Gamma$, $s'.\Phi = s.\Phi$, $\mathsf{rd}_{s'}(m,q) = \mathsf{rd}_s(m,q)$ *for all* $(m,q) \neq (n,p)$, *and* $\mathsf{rd}_{s'}(n,p) = \mathbf{0}$ *if* $u = \mathsf{nil}$ *and* $h(u)$ *otherwise.*

Proof. Directly from the effect of Operation 4.9 and Definition 3.6. □

Theorem 6.8 ($\mathsf{retarget}$)**.** *Let* $\mathrm{WF}(s)$ *and* $s \xrightarrow{\mathit{retarget}(a,d)\,/\,a'} s'$. *Put* $R = \mathsf{refs}_s(a)$ *and* $\varphi(b) := a'$ *if* $b = a$, $\varphi(b) := b$ *otherwise. Then:*

(a) $a \in \operatorname{dom} s'.\mathsf{Acc}$ *with* $s'.\mathsf{Acc}(a) = s.\mathsf{Acc}(a)$ *(the old address and datum are kept), and* $s'.\mathsf{Acc}(a') = d$.

(b) $\mathsf{rd}_{s'}(n,p) = \varphi(\mathsf{rd}_s(n,p))$ *for* all n, p; $s'.\Phi = s.\Phi$ *and* $\operatorname{dom} s'.\Gamma = \operatorname{dom} s.\Gamma$.

(c) If $a' \neq a$: $\mathsf{refs}_{s'}(a) = \emptyset$ *and* $\mathsf{refs}_{s'}(a') = \mathsf{refs}_s(a') \cup R$. *In particular, if* d *was already indexed, its class and that of* a *merge; no separate merge rule is needed.*

(d) (coarsening) $\mathsf{rd}_s(n,p) = \mathsf{rd}_s(m,q) \Rightarrow \mathsf{rd}_{s'}(n,p) = \mathsf{rd}_{s'}(m,q)$.

(e) The step is atomic: no state in which only some referrers have been moved is a state of the transition system.

Proof. (a) Lemma 6.2 and Definition 4.1. (b) Slots in R receive $a' = \varphi(a)$; all other slots keep their address $b \neq a$, so $\varphi(b) = b$; absent slots read $\mathbf{0} = \varphi(\mathbf{0})$ because $a \neq \mathbf{0}$. (c) By (b), for every address $b \neq \mathbf{0}$ we have $\mathsf{refs}_{s'}(b) = \bigcup\{\mathsf{refs}_s(c) : \varphi(c) = b\}$. If $a' \neq a$ then no c satisfies $\varphi(c) = a$ (since $\varphi(a) = a' \neq a$ and $\varphi(c) = c \neq a$ otherwise), so $\mathsf{refs}_{s'}(a) = \emptyset$; and $\varphi(c) = a'$ exactly for $c \in \{a, a'\}$, so $\mathsf{refs}_{s'}(a') = \mathsf{refs}_s(a) \cup \mathsf{refs}_s(a')$. (d) From (b), as φ is a function. (e) The effect is a single transition. □

Theorem 6.9 (Field operations are observationally conservative)**.** *Let* $\mathrm{WF}(s)$.

(a) If $s \xrightarrow{\mathit{addDef}(x,p,\tau)\,/\,\mathsf{ok}} s'$ *then* $\mathsf{rd}_{s'} = \mathsf{rd}_s$ *and* $\mathsf{eff}_{s'}(n) = \mathsf{eff}_s(n)$ *for all* n.

(b) If $s \xrightarrow{\mathit{dropDef}(x,p)\,/\,\mathsf{ok}} s'$ *then* $\mathsf{rd}_{s'} = \mathsf{rd}_s$ *and* $\mathsf{eff}_{s'} = \mathsf{eff}_s$, *and for every container* n *of* x *we have* $\mathsf{ovr}_{s'}(n) = \mathsf{ovr}_s(n) \cup \{p\}$.

Proof. (a) Padding adds only nodes with address $\mathbf{0}$, which rd already returns for absent properties (Definition 3.6) and which eff discards. Consequently an implementation may realize addDef lazily without observable difference (**[OD-3]**). (b) Contents are unchanged; by WF5 of s, $p \in \operatorname{dom} \mathsf{cnt}_s(n)$ for each container n of x. □

Theorem 6.10 (Environment independence)**.** *The relation* $s \xrightarrow{op\,/\,r} s'$ *does not depend on any file environment* fs. *Hence a change of* fs, *including the disappearance of a file, changes no component of the state; a Reference datum with null linkage persists (Lemma 6.2), and a node changes only through an explicit* update *or* $\mathsf{retarget}$.

Proof. fs occurs in none of the Operations 4.4–4.10; it occurs only in Definition 3.13. □

Remark 6.11 (Determinism). Given the same fresh identifiers and the same sel, each operation is a function of (s, op). All theorems hold for every choice.

Remark 6.12 (Relinking files). Since the datum behind a Reference address is a path, $\mathsf{retarget}(a, \langle \pi' : \rho \rangle)$ re-points every node that used the old path to the new one in one atomic step.

7 Worked example

Example 7.1. Let $\delta = \{\mathsf{cut_num} \mapsto \mathsf{CutNo},\ \mathsf{work} \mapsto \mathsf{Work},\ \mathsf{status} \mapsto \mathsf{Status}\}$, where the types are user-defined categories and all values are strings. Write $a_v := h(\langle v : \tau \rangle)$ with the category τ of the property concerned, e.g. $a_1 = h(\langle 1 : \mathsf{CutNo} \rangle)$.

1. $\mathsf{defField}(\mathsf{Cut}, \delta)$.
2. $\mathsf{write}(\mathsf{Cut}, \{\mathsf{cut_num} \mapsto \langle 1 : \mathsf{CutNo} \rangle,\ \mathsf{work} \mapsto \langle \mathrm{modelling} : \mathsf{Work} \rangle,\ \mathsf{status} \mapsto \langle \mathrm{WIP} : \mathsf{Status} \rangle\})$ creates n_1 ($\mathsf{Cand} = \emptyset$).
3. The same with $\mathsf{cut_num} = 2$ creates n_2: $\mathsf{eff}(n_1) \not\subseteq \mathsf{eff}(\overline{W})$ because $(\mathsf{cut_num}, a_1)$ is absent from the new content.
4. Repeating step 2 returns n_1 and creates nothing: $n_1 \in \mathsf{Cand}$ (Theorem 6.6(e)).
5. $\mathsf{update}(n_1, \mathsf{status}, \langle \mathrm{done} : \mathsf{Status} \rangle)$: n_1 now points to a_{done}; n_2 still points to a_{WIP}.
6. $\mathsf{update}(n_2, \mathsf{note}, \langle \mathrm{retake} : \mathsf{Note} \rangle)$: $\mathsf{note} \notin \operatorname{dom} \delta$, so it is an override: $\mathsf{ovr}(n_2) = \{\mathsf{note}\}$; reading note on n_1 yields $\mathbf{0}$.
7. $\mathsf{retarget}(a_{\mathrm{WIP}}, \langle \mathrm{in\ progress} : \mathsf{Status} \rangle)$: $\mathsf{refs}(a_{\mathrm{WIP}}) = \{(n_2, \mathsf{status})\}$ is moved to a_{prog}; a_{WIP} remains in Acc without referrers, and no record of the previous address is kept (Remark 4.12).
8. The graph node $g = (\mathsf{status}, a_{\mathrm{prog}}, \{(\mathsf{cut_num}, a_2)\})$ evaluates to $\{n_2\}$.
9. With $g_{\mathrm{done}} = (\mathsf{status}, a_{\mathrm{done}}, \{(\mathsf{cut_num}, a_2)\})$, the step $\mathsf{update}(n_2, \mathsf{status}, \langle \mathrm{done} : \mathsf{Status} \rangle)$ satisfies $\mathsf{Enter}(g_{\mathrm{done}})$, so an edge e with $\varphi_e = \mathsf{Enter}(g_{\mathrm{done}})$ fires and Rt_e may return operations, e.g. an update on a downstream cut.

8 Decision register

Table 1 lists what the author of the specification decided; these are normative. Table 2 lists points on which the source material is silent, ambiguous or in tension, together with the provisional treatment used above. Each **[OD-n]** can be changed without affecting the others unless stated.

Table 1: Decisions fixed by the author.

ID	Decision
F1	Containers are created or extended by write (subset-extension rule) and changed by update, which addresses one property of one container.
F2	Container identifiers are random and unique; they are unrelated to content.
F3	Containers hold addresses only; data live in Access; identical data (value and type) share one address obtained by hashing.
F4	Editing the datum behind an address moves all its referrers to the address of the new datum and keeps the old address (retarget); editing one node assigns it another address (update).
F5	The value replaced by an update is not retained.
F6	Graph nodes carry subnodes; evaluation returns all matches; choosing among them is left to implementations.
F7	The mechanisms of contextual existences are implementation-defined; the specification provides firing conditions for edge runtimes, while the runtime language is implementation-dependent.
F8	The properties of a field are included in the properties of every container of the field; the surplus are overrides; an absent property reads as null.

ID	Decision
F9	A lost file leaves its Reference datum intact (null linkage); nothing changes until an explicit update.
F10	Edge servers, replication and merging are implementation features.
F11	The expression of a directional edge is stored in a special expression node.
F12	Values are strings. A type is a user-defined category of data (e.g. person) with no programming-language meaning.
F13	The type of the datum behind a node must equal the type that the Field prescribes for that property; a mismatch is an error (Err) and has no effect. A null node is exempt, since the null address has no datum.

Table 2: Open decisions and their provisional treatment.

ID	Open point and provisional treatment
OD-1	**Several write candidates.** The source rule presupposes one container n_0 below the new content. If several qualify, the behaviour (absorb all, pick one by a rule, reject, create) is undetermined. *Treatment:* parameter sel (Assumption 4.7); all theorems hold for any sel.
OD-2	**Null versus absent.** A null-address node is treated as an absent node (Definition 3.7), so that padding with null does not defeat the subset test of write. *Alternative:* compare raw node sets; then filling a null slot by write would create a new container.
OD-3	**Field extension.** The source requires the field's properties to be included in each container's properties and also states the null-read rule. *Treatment:* eager padding with null in addDef, so that WF5 holds literally; Theorem 6.9 shows that a lazy realization is observationally equivalent.
OD-4	**Types of overrides.** Type conformance for properties defined by a Field is fixed (F13). A property that a container carries but its Field does not define (an override) has no prescribed type, so none is checked. *Alternative:* require every override to be declared with a type, or restrict overrides to a default category.
OD-5	**Functionality.** "Series" of definitions and nodes is read as one type per property per field and at most one address per property per container, consistent with update on a single property. Multi-valued properties would need a different update operation.
OD-6	**Unspecified lifecycle.** No deletion of containers or fields, renaming of properties, or moving of a container between fields is specified. The candidate search of write is restricted to the same field (source: "in field x_1").
OD-7	**Files.** A Reference address names a path, not content; the file environment is external (Definition 2.6). Whether the VFS overwrites files in place (so that resolve changes with no operation) or stores each save under a new path (so that a new Reference must be installed explicitly) is not specified.
OD-8	**Storage of contextual definitions.** Implementation-defined; the specification does not guarantee that they survive replication (Remark 5.12).
OD-9	**Conditions and runtimes.** "Designated conditions are satisfied" is formalized as a predicate on a step, which admits edge- and level-triggered readings. The special expression node is modelled as a slot whose datum has the reserved type ε. Scheduling, ordering, loop prevention, double execution after merges, failure handling and sandboxing are implementation-defined.

ID	Open point and provisional treatment
OD-10	**Hash function.** The source uses MD5; the specification parametrizes h and adds failure on a null hash or a collision between different data. MD5 is not collision-resistant against adversarial inputs; the choice of h is a deployment decision.
OD-11	**Status of types.** Types are read as user-defined, opaque categories. *Treatment:* $\mathcal{T}$ is the set of all labels, so there is no declaration step and no operation registers a type. Not decided: whether a category must be declared before use, and whether a category is tied to a Field (e.g. a value of type person must name an existing container of a Field Person).

A Summary of operations

Operation	changes Φ	changes Acc	changes Γ
defField(x, δ)	adds field x	—	—
addDef(x, p, τ)	adds (p, τ)	—	null padding of containers of x
dropDef(x, p)	removes p	—	— (nodes become overrides)
write(x, W)	—	adds new data	adds or replaces one container
update(n, p, u)	—	adds u if new	one slot of one container
retarget(a, d)	—	adds d if new	every slot in refs(a)

B Index of well-formedness conditions

WF1	stored data are at their own address
WF2	stored data are admissible (references are paths, expressions are programs)
WF3	containers point only to stored addresses or to null
WF4	every container belongs to an existing field
WF5	field properties are included in container properties (surplus = overrides)
WF6	non-null nodes of defined properties have the defined type